

0

Course Introduction

소프트웨어 Architecture 란 무엇이며 왜 중요한가, 그리고 이 과정에서 무엇을 배우는가

- Background · Software Architecture 정의
- Why is Software Architecture Important?
- Course Objectives · Topics

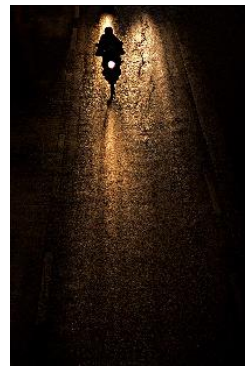
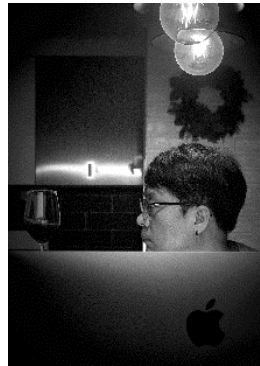
강사 소개



Gemini

• 김덕겸 : 설비 SW Architect 그룹 / DTC 센터

- S/W Architect 인증 ('14~)
- TRIZ Specialist Level 3 ('05~)
- CTO SW Architecture 강의
- SSIT 설비 시스템 개발 강의
- Associate Architect 과정 전문강사 (설계, 평가)



강의 시간표

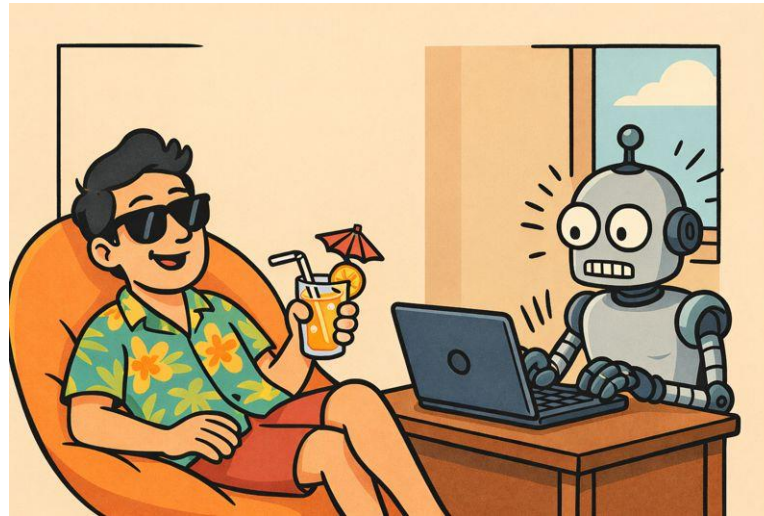
시간	1일차	2일차
9:30	과정 소개	Ex 2 발표 및 피드백
10:00		1.7-2 Distributed Architecture (26min) 1.8-1 Deployment Architecture (20min) 1.8-2 Deployment Architecture (26min)
10:30	0. Course Introduction (28min) 1.1 Introduction to Architecture Style (25min) 1.2 Data Flow Architecture (33min) 1.3 Data Centered Architecture (18min) 1.4 Implicit Invocation Architecture (19min)	
11:00		
11:30		Ex 3. Architecture Style 실습 3 (DsA, DpA) - 9 styles
12:00		
12:30	점심 식사	점심 식사
13:00		
13:30	실습 소개	Ex 3 발표 및 피드백
14:00	Ex 1. Architecture Style 실습 1 (DFA, DCA, IIA) - 7 styles	2.1-1 Architecture Views (22min) 2.1-2 Architecture Views (24min) 2.2-1 Documenting Architecture using SEI (42min) 2.2-2 Documenting Architecture using SEI (47min) 2.3 Documenting Architecture using Hierarchical ~Summary (30min)
14:30		
15:00	1.5 Interaction-Oriented Architecture (31min) 1.6-1 Hierarchical Architecture (25min) 1.6-2 Hierarchical Architecture (26min) 1.7-1 Distributed Architecture (26min)	
15:30		
16:00		
16:30		
17:00	Ex 1 발표 및 피드백	Ex 4. Architecture View 실습 1 (Documenting Top Level View)
17:30	Ex 2. Architecture Style 실습 2 (IOA, HA) - 7 styles	
18:00		

강의 시간표

시간	3일차	4일차
9:30	Ex 4 발표 및 피드백	3.10 Testability Tactics (17min) / 3.11 Usability Tactics (12min) 3.12 Tactics and Architectural Patterns (21min)
10:00	Ex 5. Architecture View 실습 2 (Documenting Component Level View)	
10:30		Ex 6 발표 및 피드백
11:00		Ex 7. Architecture Tactic 실습 2 (Performance~Usability 중심설계)
11:30	3.1 Architectural Tactics (17min) 3.2 Availability Tactics (38min)	
12:00		
12:30	점심 식사	점심 식사
13:00		
13:30	3.3 Deployability Tactics (27min) 3.4 Energy Efficiency Tactics (13min) 3.5 Integrability Tactics (27min) 3.6 Modifiability Tactics (14min)	Ex 7 발표 및 피드백
14:00		4-1 Architecture Design (22min) 4-2 Architecture Design (34min)
14:30		
15:00		Ex 5 발표 및 피드백
15:30	Ex 6. Architecture Tactic 실습 1 (Availability~Modifiability 중심 설계)	
16:00		
16:30		
17:00		
17:30	.7 Performance Tactics (16min) 3.8 Safety Tactics(17min) / 3.9 Security Tactics (17min)	Ex 8 발표 및 피드백
18:00		과정 정리

AI 시대!

우리는 왜 Architecture를 공부하는 걸까요?



Vibe Coding



Andrej Karpathy

개발자가 대규모 언어 모델(LLM)에 프로젝트나 작업을 설명하면 LLM이 프롬프트에 따라 소스 코드를 생성하는 방식

"**분위기**에 완전히 몰입하고, 기하급수적 증가를 수용하고, 코드가 존재한다는 사실조차 잊어버리는 것"

Vibe Coding의 한계 - 왜 실패하는가?

VIBE CODING

1st hour



VIBE CODING

11th hour



Context Depth

‘부분적 성공(Vibe)이 쌓여, 전체적 불투명성(Debt)을 만든다’
길어지면 앞의 결정을 잊는다.

아무도 모르는 상태 (Black Box Problem)

무엇이 왜 그렇게 되어 있는지 설명할 수 없다

매번 조금씩 다르다 (일관성 결여)

동일한 요청에도 항상 동일한 결과를 제공하지 않는다
고쳤던 게 다시 망가진다

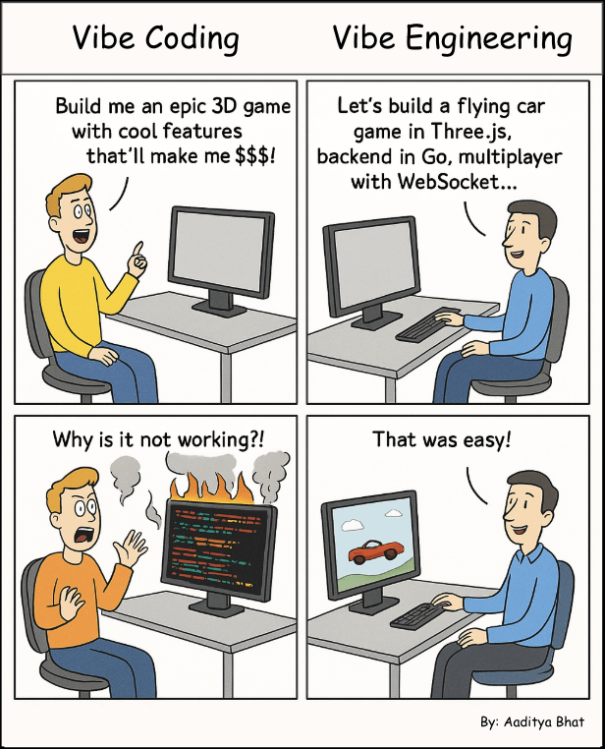
뒤늦게 드러난다

알려진 문제일 때는 안 보이다가, 낫설어지면 바로 드러난다

Vibe Engineering <https://simonwillison.net/2025/Oct/7/vibe-engineering/>

'Vibe Coding은 AI를 이용해 빠르고, 허술하고, 무책임하게 소프트웨어를 개발하는 방식' - Simon Willison -

우리가 Architecture 설계를 알아야 하는 이유!



Context Depth

아무도 모르는 상태
(Black Box Problem)

매번 조금씩 다르다
(일관성 결여)

뒤늦게 드러난다

사전 계획

전체 Context 제공
작업 Context 분리

포괄적 문서화

정리된 요구사항/구조설계 문서
→ Guard Rail

버전 관리

실수를 빠르게 복원
반복/변경의 영향 최소화

자동화된 테스트
+ 수동 QA

초기에 Risk를 확인
'코드를 읽어서 판단하지 않는다'

"문서 쓸 일만 늘어난 거 아닌가요?"

코드는 Agent가 짜주는데, 대신 문서 작업은 늘었습니다. 이러면 편해지는 게 없는 것 같은데요?



'Vibe' Engineering

Vibe를 버리는 게 아니라, Vibe가 무너지지 않도록 받치는 것

코딩에서 그랬듯이
그 문서들도 Agent가 쓰게 하면 됩니다.

토의한 내용을 Agent가 문서로 정리한다

단, 하나는 사람이!

어쨌든, 우리가 Architecture 설계를 알아야 하는 이유!
검증은 여러분이 합니다 — 읽고, 고치고, 승인하는 것.

모르면 이렇게 됩니다...

Background: 개집에서 고층 빌딩까지

■ 개집(dog house)을 짓는다면

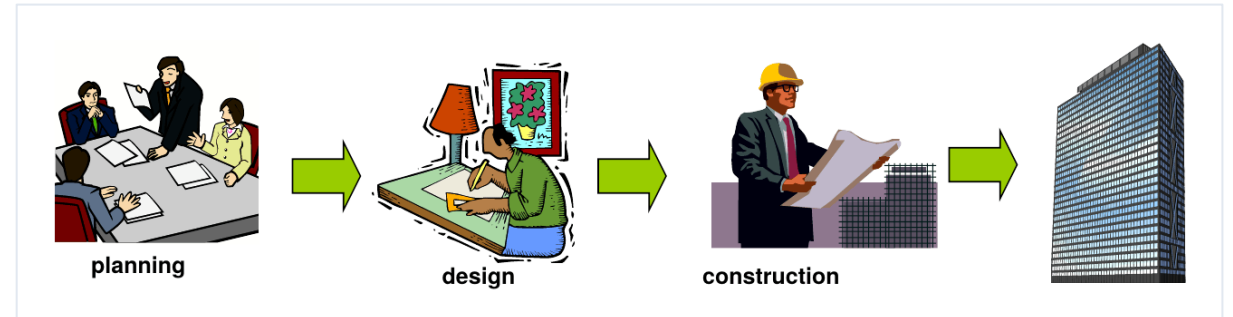
- 나무를 자르고 못을 박는 것만으로 충분히 만들 수 있다

■ 2층 집을 짓는다면

- 더 많은 인력과 더 많은 도구가 필요하다
- 짓기 전에 설계도(blue-print)를 그려야 할 수도 있다

■ 고층 빌딩을 짓는다면

- 수많은 전문가가 참여한다
- 치밀한 계획(planning)과 설계(design) 없이는 지을 수 없다
- Architecture 가 건축 성공의 열쇠이다



고층 빌딩: *planning → design → construction*

소프트웨어에 대한 비유(Analogy)

■ 규모가 커질수록 구조가 중요해진다

- 소프트웨어 시스템의 크기와 복잡도가 커지면 전체 시스템 구조, 즉 Software Architecture 의 설계가 더 중요한 문제가 된다
- 알고리즘이나 자료구조의 선택보다 구조적 문제가 더 큰 이슈가 된다

■ 구조적 문제(structural issues)의 예

- 시스템을 Component 들의 조합으로 어떻게 구성할 것인가
- Component 사이의 통신 프로토콜
- 기능(functionality)을 어떤 설계 요소에 배정할 것인가
- 물리적 분산(physical distribution) 등

핵심 고층 빌딩에 설계도가 필요하듯, 큰 소프트웨어에는 Architecture 가 필요하다

Software Architecture 의 정의

■ 정의

- 다양한 관점에서 소프트웨어의 구조와 운영을 위한 여러 설계를 도출하고, 그중 최적의 설계를 선정 및 통합한 결과.
(프로그램 또는 컴퓨팅 시스템의 구조 또는 구조들의 집합)

Software Architecture 다음으로 이루어진다

- 소프트웨어 요소(software elements)
- 요소들의 외부에서 보이는 속성(externally visible properties)
- 요소들 사이의 관계(relationships)

■ "외부에서 보이는 속성"이란

- 다른 요소가 그 요소에 대해 가정할 수 있는 것
- 예: 요소가 제공하는 서비스, 성능 특성 등

핵심 Architecture = 요소 + 요소의 외부 속성 + 요소 간 관계

정의가 의미하는 것(Implications)

■ 모든 소프트웨어 시스템은 Architecture 를 가진다

- 명시적으로 설계했든 아니든, 소프트웨어가 있는 컴퓨팅 시스템에는 반드시 Architecture 가 존재한다

■ 시스템은 하나 이상의 구조를 가질 수 있다 *(다양한 관점에서 소프트웨어의 구조와 운영을 위한 여러 설계를 도출...)*

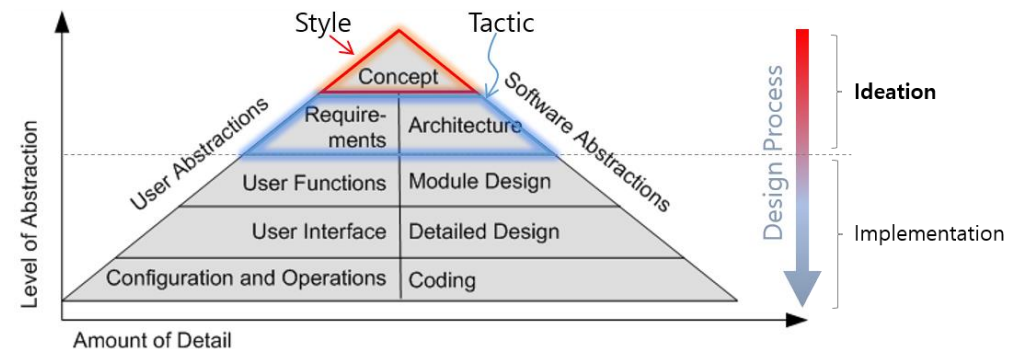
- Module 구조, 실행 시간(run time) 구조, 할당(allocation) 구조 등

■ Architecture 는 행위(behavior)를 포함한다

- 다른 요소의 관점에서 알아볼 수 있는 행위라면 각 요소의 행위도 Architecture 의 일부이다

■ Architecture 는 추상화(abstraction)이다

- 요소 내부의 세부 정보(local information)는 감춘다



- SCOTTALTHAM.COM -

왜 Software Architecture 가 중요한가?

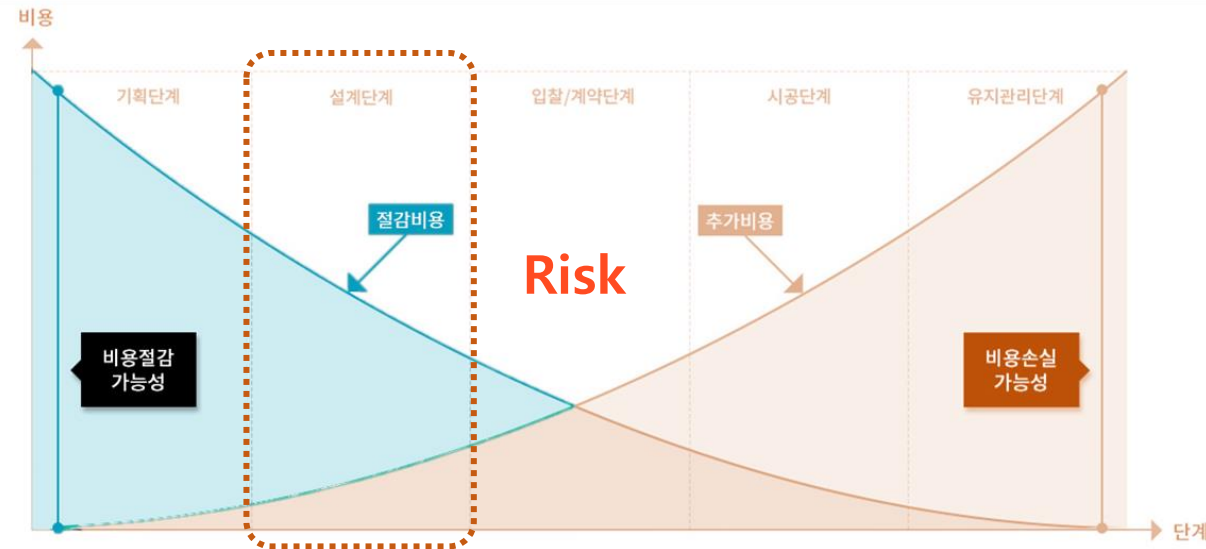
■ 세 가지 이유

- 시스템의 품질을 결정한다 (*Quality Attribute*를 억제하거나 가능하게 한다)
- 시스템의 이전 가능한(transferable) 추상화이다
- Stakeholder(이해관계자) 사이의 의사소통 수단이다

1. 시스템의 품질을 결정 한다

■ 시스템이 원하는 품질 속성을 달성할 수 있는지는 Architecture 가 상당 부분 결정한다

- 높은 Performance 가 필요하면 요소들의 시간 기반 행위를 관리하는 데 집중해야 한다
- Modifiability 가 중요하면 대부분의 변경이 소수의 요소에만 영향을 주도록 책임을 배정해야 한다
- 품질 속성은 설계·구현·배포 전 과정에서 고려해야 하며, Architecture 설계의 시작부터 고려해야 한다
- 가장 이른 Architecture 결정일수록 나중에 바꾸기 가장 어렵다



2. 이전·재사용 가능한 모델로서의 Architecture

■ Architecture 재사용

- 하나의 Architecture 를 여러 시스템에 재사용하면, 초기의 Architecture 결정들도 함께 이전되어 재사용된다

■ Architecture 재사용을 위한 설계 개념

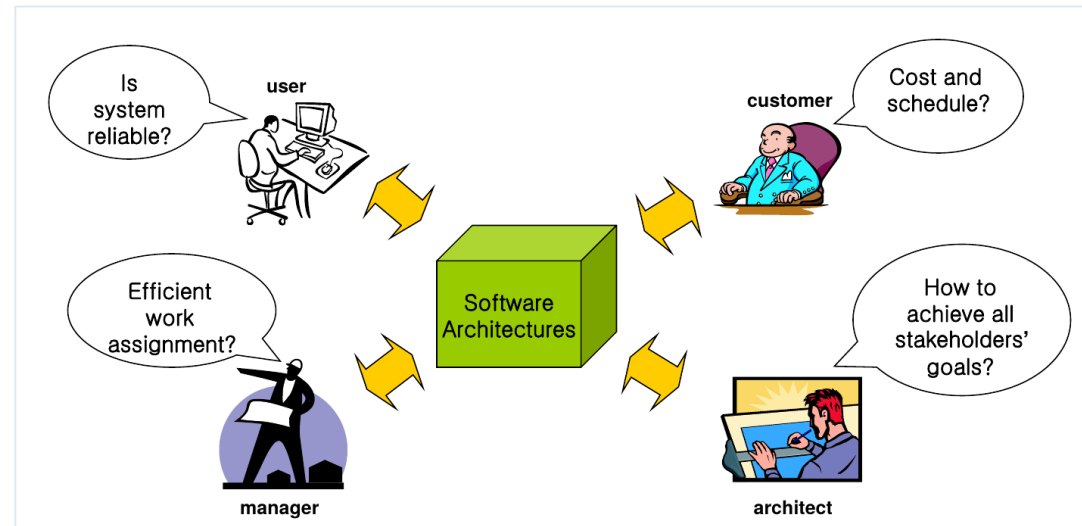
- Style: 반복해서 나타나는 시스템 구조와 패턴
- Tactic: 특정 Quality Attribute 를 달성하기 위해 쓰이는 공통 해법
- Reference Architecture: 특정 유형의 응용을 만들기 위한 Architecture 지침을 담은 표준 모델
- Framework: 응용을 만드는 기반이 되는 소프트웨어 요소들의 구조화된 집합
- Product Line Architecture: 소프트웨어 제품 라인의 제품군을 위한 재사용 가능한 Architecture

3. Stakeholder 의사소통의 수단

■ Architecture 는 이해관계자 간의 소통을 지원한다

- User: 시스템이 신뢰할 만한가?
- Customer: 비용과 일정은?
- Manager: 효율적인 작업 배정은?
- Architect: 모든 이해관계자의 목표를 어떻게 달성할까?
- 서로 다른 관심사를 가진 사람들이 Architecture 를 '공통 언어'로 삼아 논의한다

※ Remember! - Architecture 는 추상화(abstraction)이다



Software Architecture 를 중심으로 한 이해관계자의 관심사

Good Architecture?

Remind! '시스템이 원하는 품질 속성을 달성할 수 있는지는 Architecture 가 상당 부분 결정한다'

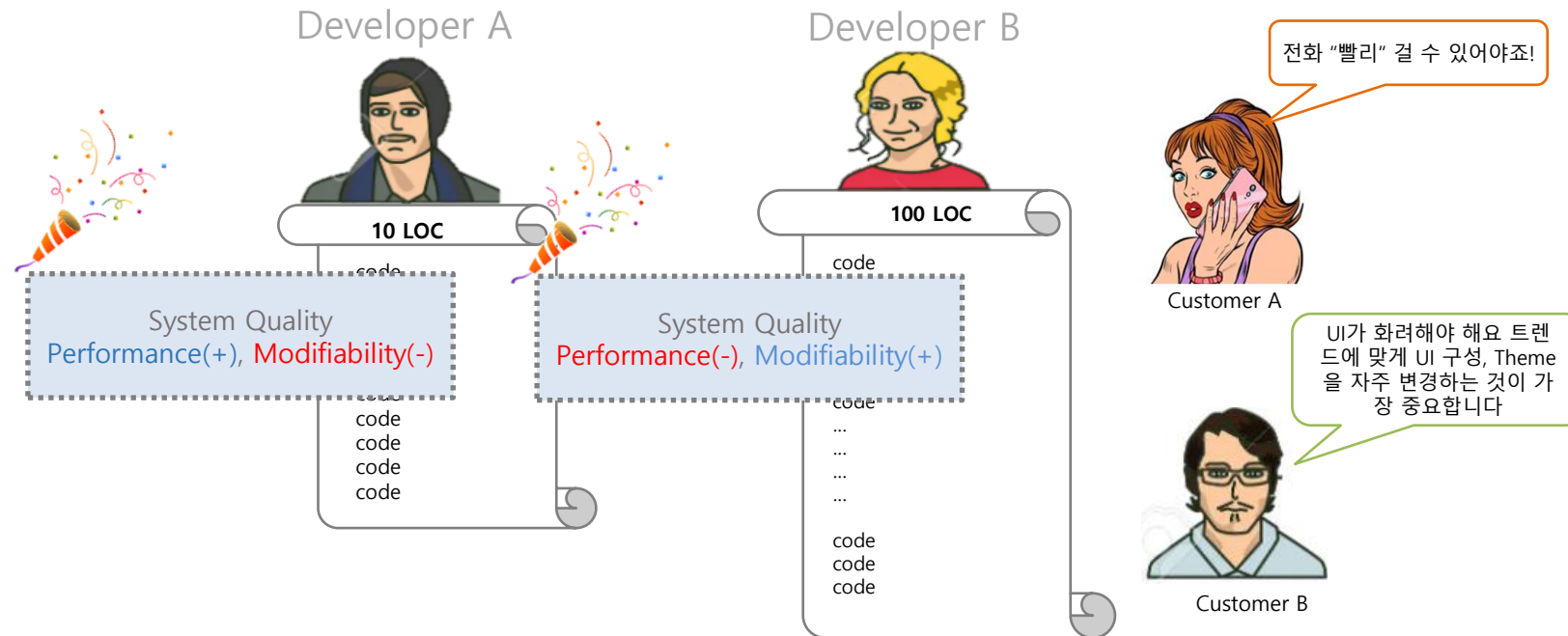


Right Architecture!

1. 시스템의 품질을 결정 한다

시스템이 원하는 품질 속성 → 고객(이해관계자)의 요구사항을 만족 시킬 수 있어야 한다.

Smart Phone "Calling function" Implementation



Course Objectives: 과정 목표 → Right Architecture의 설계

■ 이 과정을 마치면

- 소프트웨어 개발에서 Software Architecture 의 기본 개념을 이해한다
- 주요 Architectural Style 과 Pattern 을 알고, 그것이 Quality Attribute 에 미치는 영향을 분석할 수 있다
- 특정 Quality Attribute 를 만족시키는 다양한 Tactic 을 이해한다
- 문제에 대한 Architecture 대안을 만들고, 근거(rationale)를 가지고 그중 하나를 선택할 수 있다
- 여러 관점(viewpoint)에서 Architecture 를 표현할 수 있다

이 과정에서 다루는 주제

■ Architectural Styles

- 23개 주요 Architectural Style 을 configuration, example, strength, weakness 관점에서 설명한다

■ Architectural Views

- 다양한 관점의 Architectural View 각각에 대해 특징, 용도, 만드는 방법, 작성 예제 등을 설명한다

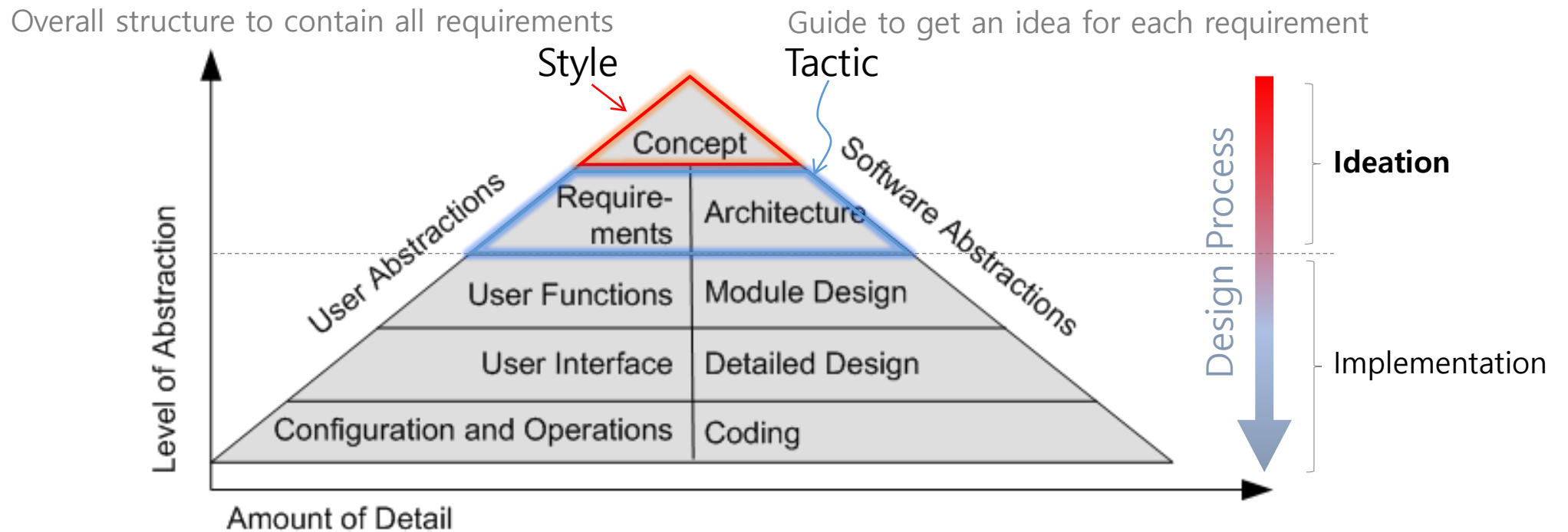
■ Architectural Tactics

- Quality Attribute 달성을 위한 10개 범주의 Tactic 을 설명한다
- Pattern 과 Tactic 을 이용한 Architecture 설계의 best practice 를 다룬다

Architectural Design Process

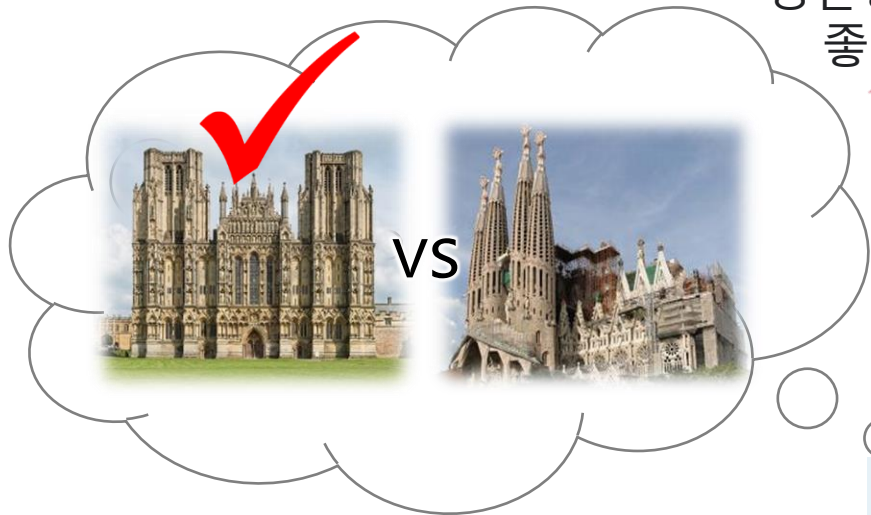
From 'Abstraction' To 'Concrete'

From 'Simple' To 'Detail'



Overall structure to contain all requirements

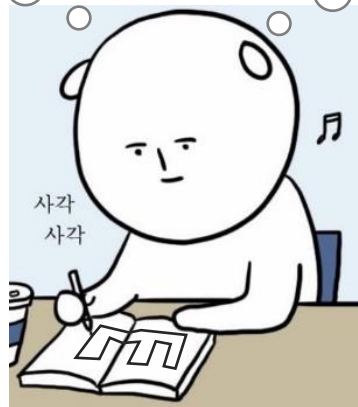
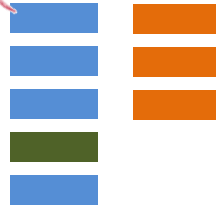
Style



이것 저것 ASR을 따져보니...
공간 활용이 용이한
고딕 스타일이 좋겠군!

냉난방 효율이
좋아야 함!

Requirements



Guide to get an idea for each requirement

Tactic

외기 차단

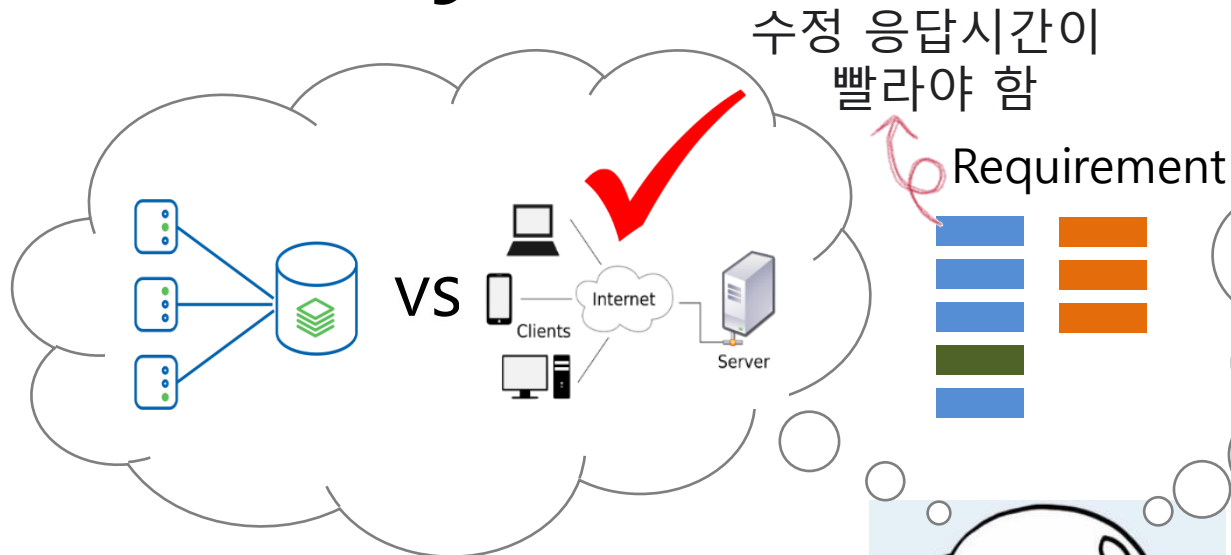


냉난방을 위해서는...
외기 차단 효과가 큰
'회전문' 으로!

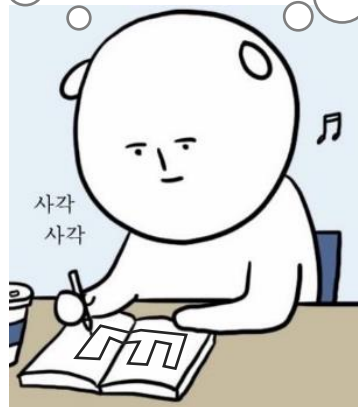


Overall structure to contain all requirements

Style

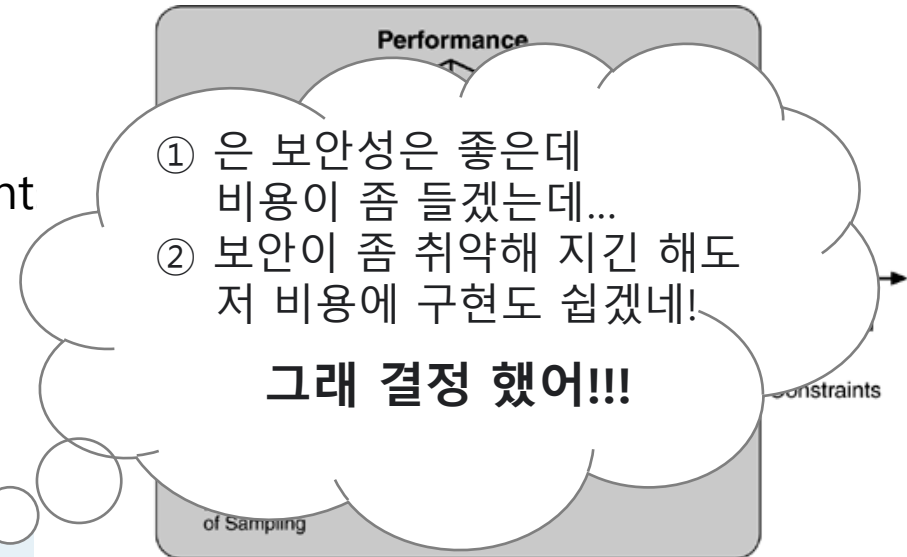


이것 저것을 따져보니...
서비스 운영이 용이한
Server-Client 스타일이 좋겠군!



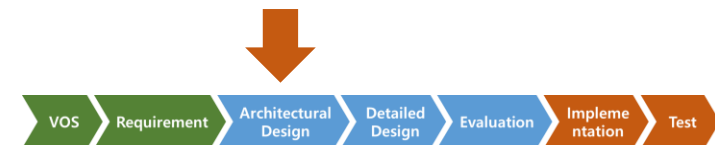
Guide to get an idea for each requirement

Tactic



수정 응답시간을 위해서는...

- ① Increase Computation Efficiency... Network Traffic을 분산해볼까?
- ② Maintain Multiple Copies... Local에 문서를 저장 할까? ✓



이전 버전과 달라진 점

■ 기반 교재

- 이전 버전: Software Architecture in Practice 3rd Edition
- 현재 버전: Software Architecture in Practice 4th Edition

■ Architectural Style 주제의 변경

- AI 관련 시스템에서의 스타일 적용 예 추가
 - Batch Sequential, Pipe and Filter, Blackboard, Event-Based Style
- 클라우드 기반 시스템에서의 스타일 적용 예 추가
 - Virtual Machine Style, Dispatcher Style
- MVC 관련 스타일의 개념과 예제 수정
- Deployment Architecture Style 추가
 - Container-Based, Cloud-Based, Edge-Based Style

Architectural View · Tactic 주제

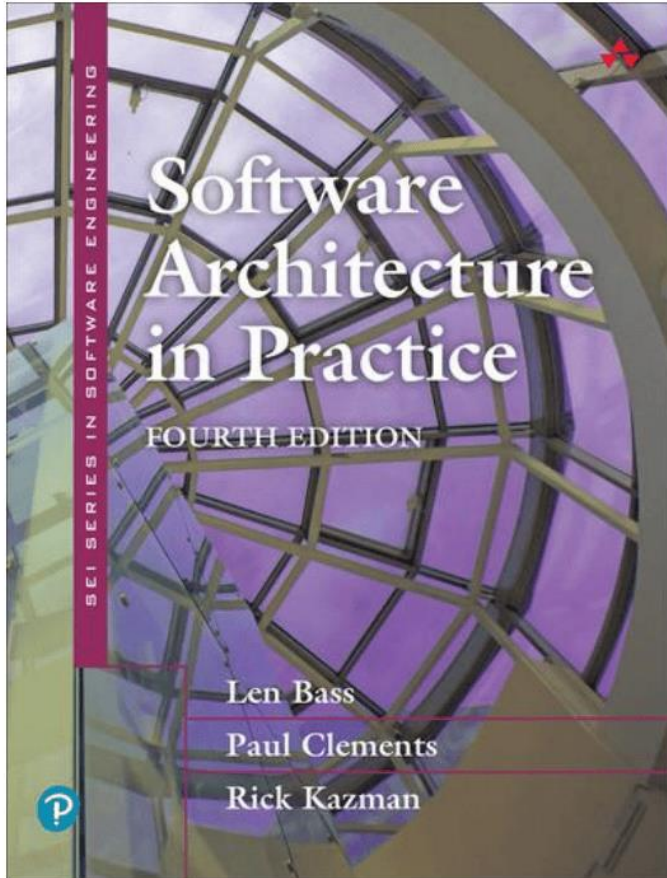
■ Architectural View 주제

- Architectural View 소개: 4+1 View Model, Siemens View Model, SEI View Model
- Architecture 문서화(Documenting Architecture)
 - SEI View Model 을 이용한 문서화
 - Hierarchical Multi-View Model 을 이용한 문서화 (4판에서 추가)
 - C4 Model 을 이용한 문서화 (4판에서 추가)

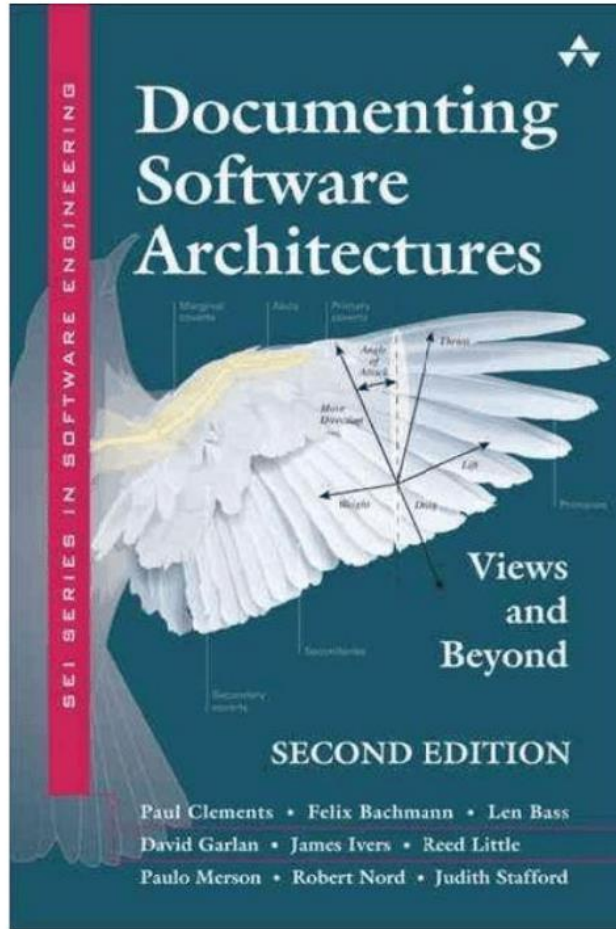
■ Architectural Tactic 주제

- Architectural Tactic 소개
- 10가지 Tactic: Availability, Deployability, Energy Efficiency, Integrability, Modifiability, Performance, Safety, Security, Testability, Usability
 - Deployability, Energy Efficiency, Integrability, Safety 는 4판에서 추가
- Tactic 과 Architectural Pattern
- Tactic 과 Pattern 을 이용한 Architecture 설계

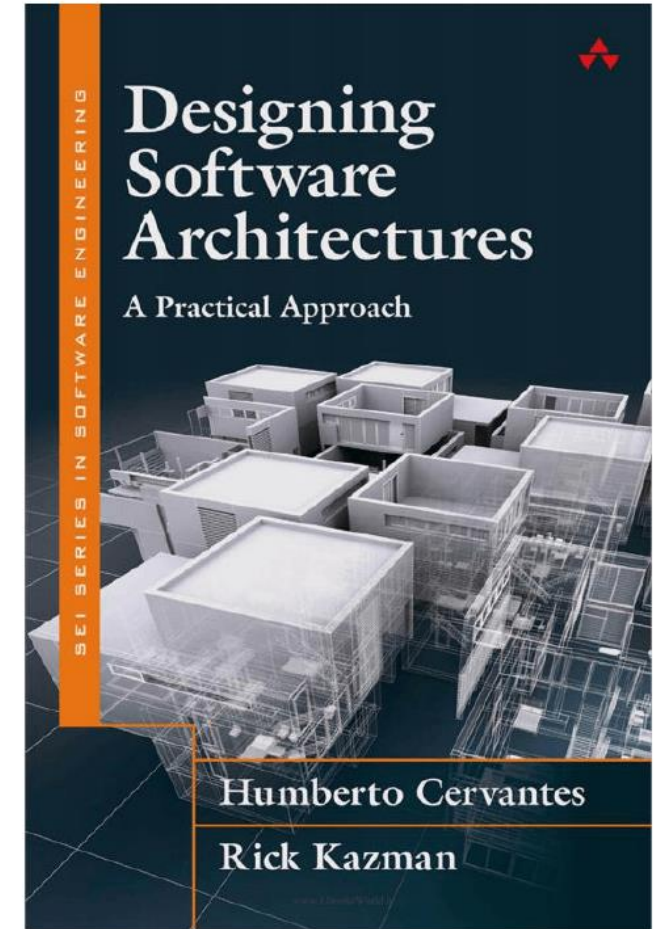
References: 참고 문헌



Software Architecture in Practice, 4th Ed. (Bass, Clements, Kazman)



Documenting Software Architectures: Views and Beyond, 2nd Ed. (Clements 외)



Designing Software Architectures: A Practical Approach (Cervantes, Kazman)